Jenkins CI/CD Lab — 5 Beginner Pipeline Projects (Windows Agent)

Course: ISWE406L — Agile Development Process and DevOps All five projects share **Stage 1: Checkout from GitHub**. Stages 2–3 differ per project.

Prerequisites (set up once, before class)

1. **Jenkins** installed and running on a Windows machine, with the **Git plugin** installed.
2. **Python** installed on the Jenkins agent, added to PATH (verify with `python --version` in cmd).
3. **flake8** and **pytest** pre-installed on the agent so class time isn't lost to installs:

   ```
   pip install flake8 pytest
   ```

4. Each student has a **GitHub account** and a **public repository** for their project.
5. In Jenkins: **New Item → Pipeline**, and either:
   - Paste the Jenkinsfile directly into the **Pipeline script** box, or
   - Select **Pipeline script from SCM** and point it at the student's GitHub repo (if they commit the `Jenkinsfile` to the repo root).

PROJECT 1: Build & Test Pipeline

Concept taught: basic CI — install dependencies, then run automated tests.

Files to create in the GitHub repo

app.py

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

test_app.py

```python
from app import add, subtract

def test_add():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(5, 3) == 2
```

requirements.txt

```
pytest
```

Jenkinsfile

```
pipeline {
    agent any
```

```
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'pip install -r requirements.txt'
            }
        }

        stage('Run Unit Tests') {
            steps {
                bat 'pytest test_app.py'
            }
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py`, `test_app.py`, `requirements.txt`, and `Jenkinsfile` to it, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo (or paste the Jenkinsfile directly).
3. Click **Build Now**.
4. Open **Console Output** — confirm all 3 stages run in order and both tests pass (`2 passed`).
5. **Try it broken:** change `add` to return the wrong value, push, and rebuild — show that the "Run Unit Tests" stage now fails and the pipeline stops there.

PROJECT 2: List Utilities — Build & Test Pipeline (with Parametrized Tests)

Concept taught: still the Build & Test pattern from Project 1, but introduces **parametrized tests** — a pytest feature that runs the same test logic against several different inputs, instead of writing a separate test function for each case. This is the natural next step once students are comfortable with basic `assert` tests.

Files to create in the GitHub repo

`app.py`

```
def find_max(numbers):
    return max(numbers)

def count_evens(numbers):
    return len([n for n in numbers if n % 2 == 0])
```

`test_app.py`

```
import pytest
from app import find_max, count_evens

@pytest.mark.parametrize("numbers, expected", [
    ([1, 5, 3], 5),
```

```
    ([-10, -2, -7], -2),
    ([4, 4, 4], 4),
])
def test_find_max(numbers, expected):
    assert find_max(numbers) == expected

@pytest.mark.parametrize("numbers, expected", [
    ([1, 2, 3, 4], 2),
    ([1, 3, 5], 0),
    ([2, 4, 6, 8], 4),
])
def test_count_evens(numbers, expected):
    assert count_evens(numbers) == expected
```

requirements.txt

```
pytest
```

Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'pip install -r requirements.txt'
            }
        }

        stage('Run Unit Tests') {
            steps {
                bat 'pytest test_app.py -v'
            }
        }
    }
}
```

Note the `-v` (verbose) flag — with parametrized tests, this is worth adding so the console output lists every individual input case that ran (e.g. `test_find_max[numbers0-5]`), rather than collapsing them into one line.

Execution procedure

1. Create a new GitHub repo, add `app.py`, `test_app.py`, `requirements.txt`, and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Open **Console Output** — point out that pytest ran **6 tests total**, even though only 2 test functions were written — each parametrized case counts as its own test.

5. **Try it broken:** add one more case to the `find_max` parametrize list with a wrong expected value (e.g. `([1, 5, 3], 999)`), push, and rebuild — show that only that specific case fails, while the others still pass. This is a good moment to explain that parametrization gives fine-grained, per-input feedback.

PROJECT 3: Manual Approval Gate — Deploy Pipeline

Concept taught: this one is *not* the Build & Test pattern — it introduces a completely different, very common real-world CI/CD idea: a **manual approval gate**. Many pipelines shouldn't deploy automatically the moment code is checked in — someone should look and click "yes, go ahead" first. Jenkins supports this natively with the `input` step, which literally pauses the pipeline mid-run and waits for a human.

Files to create in the GitHub repo

`app.py`

```
print("Deploying application version 1.0...")
print("Deployment complete.")
```

`Jenkinsfile`

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                echo 'Build successful: app.py compiled with no syntax errors'
            }
        }

        stage('Deploy') {
            steps {
                input message: 'Approve deployment to production?', ok: 'Deploy'
                bat 'python app.py'
            }
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Watch the pipeline reach the **Deploy** stage and then simply **stop moving** — no error, no crash, it's just waiting. In the Jenkins UI (either the build's console output or the pipeline visualization), a prompt appears: *"Approve deployment to production?"* with **Deploy** and **Abort** buttons.

5. Click **Deploy** — the pipeline resumes, runs `python app.py`, and finishes successfully.
6. **Try the other path:** run the build again, and this time click **Abort** instead. Show that the pipeline stops there and is marked as **aborted**, and `app.py` never actually runs — nothing gets "deployed" unless a human explicitly approved it.
7. Discuss with students: this is exactly how real deployment pipelines protect production systems — automated stages (checkout, build, test) run freely, but the moment something risky or irreversible is about to happen (like deploying to real users), a human checkpoint is inserted on purpose.

PROJECT 4: Environment Variables + Linting Pipeline

Concept taught: Jenkins auto-injects built-in environment variables (`BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE`) into every run.

Files to create in the GitHub repo

app.py

```python
def greet(name):
    return "Hello, " + name
```

Jenkinsfile

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Show Build Info') {
            steps {
                echo "Build Number: ${env.BUILD_NUMBER}"
                echo "Job Name: ${env.JOB_NAME}"
                echo "Workspace: ${env.WORKSPACE}"
            }
        }

        stage('Run Linter') {
            steps {
                bat 'flake8 app.py'
            }
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**, then click it again (**Build Now** a second time).

4. Open **Console Output** for both builds and compare — `BUILD_NUMBER` increases (e.g. `#1` then `#2`) while `JOB_NAME` and `WORKSPACE` stay the same. This shows Jenkins tracks build identity automatically.
5. Since `app.py` above is clean, "Run Linter" should pass. **Try it broken:** add a line with a trailing space or an unused import, push, rebuild — show that flake8 now fails the stage.

PROJECT 5: Post-Build Success/Failure Pipeline

Concept taught: the `post` block — pipelines can react differently depending on whether the build passed or failed. This is separate from the numbered `stages`.

Files to create in the GitHub repo

`app.py` (a clean, compilable file)

```python
def multiply(a, b):
    return a * b

print(multiply(4, 5))
```

`Jenkinsfile`

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Compile Check') {
            steps {
                bat 'python -m py_compile app.py'
            }
        }
    }

    post {
        success {
            echo 'Build succeeded: app.py has no syntax errors.'
        }
        failure {
            echo 'Build failed: check app.py for syntax errors.'
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now** — confirm the build is marked **success** (green) and the console shows the "Build succeeded" message from the `post` block.

4. **Try it broken:** introduce a syntax error in `app.py` (e.g. remove a closing parenthesis: `print(multiply(4, 5)`), push, and rebuild.
5. Confirm the build is marked **failed** (red) and the console shows the "Build failed" message instead — demonstrating that `post` runs regardless of outcome, but chooses the block matching what happened.

Summary table

	Concept	Stage 2	Stage 3 / post
Project 1	Automated testing (basics)	Install dependencies	Run unit tests
Project 2	Parametrized tests (multiple cases, one test function)	Install dependencies	Run unit tests (verbose)
Project 3	Manual approval gate before deploy	Build (compile check)	Deploy (with `input` approval)
Project 4	Built-in environment variables	Show build info	Run linter
Project 5	Post-build success/failure hooks	Compile check	`post { success / failure }`

Note on Project 5: it has only 2 stages inside the `stages` block, plus a `post` block — worth explicitly telling students that `post` is a lifecycle hook, not a numbered stage.

BEGINNER'S PRACTICAL GUIDE

Kubernetes Integration in a CI/CD Pipeline

A step-by-step hands-on lab using VS Code, GitHub, GitHub Actions, Docker, and Kubernetes

Stage	What students will do
Source code	Create a simple Python Flask application in VS Code
Version control	Commit and push code to GitHub
Continuous Integration	Automatically check out code, install dependencies, and run tests
Docker	Build a container image and publish it to Docker Hub
Kubernetes	Deploy the image, expose the app, and update the running version

Recommended approach: Complete the local application, testing, and Docker steps first. Then connect GitHub Actions. Finally, deploy to Docker Desktop Kubernetes. This avoids overwhelming first-time learners.

Suggested duration: two practical sessions of approximately 2–3 hours each.

1. Learning objectives and pipeline overview

By the end of this lab, students should be able to create and test a small web application, push code to GitHub, read a GitHub Actions run, build and publish a Docker image, and deploy that image to Kubernetes.

Tool	Purpose
VS Code	Write and manage project files
Git	Track changes and create commits
GitHub	Host the source repository
GitHub Actions	Automate checkout, testing, and image publishing
Docker	Package the application and dependencies into an image
Docker Hub	Store the published image
Kubernetes / kubectl	Run, update, and inspect the application

Pipeline flow

VS Code → Git commit/push → GitHub → GitHub Actions checkout → install dependencies → run tests → build/push Docker image → Kubernetes Deployment and Service.

Important: Git manages source history; GitHub hosts the repository; GitHub Actions automates tasks; Docker packages the app; Kubernetes manages the running containers.

2. Prerequisites and installation checks

Before the practical, ask each student to install and verify the following:

- Visual Studio Code.
- Git for Windows (Git Bash is useful, or use PowerShell).
- A GitHub account.
- Docker Desktop, with Kubernetes enabled and running.
- Python 3.12 or a compatible Python 3 version.
- kubectl, the Kubernetes command-line tool.
- A Docker Hub account, with a public repository for this introductory exercise.

Official setup links:

- VS Code download
- Git downloads
- Docker Desktop
- Docker Desktop Kubernetes
- Install kubectl
- Docker Hub

Open VS Code → Terminal → New Terminal. Run these commands one at a time:

```
git --version
python --version
docker --version
docker info
kubectl version --client
```

```
kubectl get nodes
```

Expected: each command runs successfully, and kubectl get nodes shows a node with status Ready. Do not proceed until Docker works and Kubernetes is Ready.

3. Create the sample Flask application

Step 1 — Create the project folder

Create a folder named student-k8s-demo and open it in VS Code. Create this structure:

```
student-k8s-demo/
■■■ app.py
■■■ requirements.txt
■■■ test_app.py
■■■ Dockerfile
■■■ .dockerignore
■■■ .gitignore
■■■ k8s/
■   ■■■ deployment.yaml
■   ■■■ service.yaml
■■■ .github/
    ■■■ workflows/
        ■■■ pipeline.yml
```

Step 2 — Create app.py

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello Students! Kubernetes CI/CD is working!"

@app.route("/health")
def health():
    return jsonify(status="healthy"), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

The home route displays a message. The /health route returns a simple health status. The application listens on port 5000.

Step 3 — Create requirements.txt

```
Flask>=3.0,<4.0
pytest>=8.0,<9.0
```

Step 4 — Create test_app.py

```
from app import app

def test_home_page():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200
    assert b"Kubernetes CI/CD is working" in response.data

def test_health_endpoint():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json["status"] == "healthy"
```

Step 5 — Run locally

```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
# Git Bash alternative:
# source .venv/Scripts/activate

python -m pip install -r requirements.txt
python -m pytest -v
python app.py
```

Step 5 — Run locally in Windows VS Code

Windows PowerShell may block the virtual-environment activation script with an error saying that running scripts is disabled. This does not necessarily mean the virtual environment failed to create. Use either option below from the student-k8s-demo project folder.

Option 1 — Use the virtual environment without activating it (recommended)

Run each command in the VS Code PowerShell terminal. This method does not require changing the PowerShell execution policy.

1. Install the packages:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

2. Run the automated tests:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Expected result: both tests pass. If a test fails, read the error message and check app.py and test_app.py.

3. Start the Flask application:

```
.\.venv\Scripts\python.exe app.py
```

Keep this terminal open while testing. Open these addresses in a browser:

```
Home page:     http://127.0.0.1:5000/
Health check:   http://127.0.0.1:5000/health
```

The home page should show the welcome message. The health endpoint should return {"status":"healthy"}. Stop the server with Ctrl+C in the terminal.

Option 2 — Allow activation for the current PowerShell session

Use this option only if you want to activate the virtual environment normally and your institution does not prohibit it. This setting applies only to the current PowerShell process and is discarded when that terminal session closes.

1. Enter:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

If PowerShell asks for confirmation, follow the prompt. Then activate the environment:

```
.\.venv\Scripts\Activate.ps1
```

The terminal prompt should now begin with (.venv), for example:

```
(.venv) PS D:\Official - VIT\2026-27\Academics\Agile and DevOps\student-k8s-demo>
```

Then run:

```
python -m pip install -r requirements.txt
python -m pytest -v
python app.py
```

If your organization enforces an execution policy, do not attempt to override it. Use Option 1 instead.

Checkpoint: the app opens in the browser, both tests pass, and you can stop the server with Ctrl+C. Once this works, continue to the Dockerfile and image-build steps.

Open http://127.0.0.1:5000/ and http://127.0.0.1:5000/health in a browser. Stop the server with Ctrl+C. If PowerShell blocks virtual-environment activation, use Git Bash or follow the institution's approved PowerShell policy.

Checkpoint 1: The app runs and both tests pass before proceeding.

4. Build and run the Docker image

Step 6 — Create Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 7 — Create .dockerignore

```
.venv/
__pycache__/
.pytest_cache/
.git/
```

Step 8 — Build the image

```
docker build -t student-k8s-demo:v1 .
docker images
```

The dot at the end tells Docker to use the current directory as the build context.

Step 9 — Run and test the container

```
docker run --rm -d --name student-demo -p 5000:5000 student-k8s-demo:v1
docker ps
docker logs student-demo
```

Open http://localhost:5000/. Stop the container with:

```
docker stop student-demo
```

If host port 5000 is already occupied, use -p 5001:5000 and open http://localhost:5001/.

Checkpoint 2: Students can build the image, run a container, open the app, inspect logs, and stop the container.

5. Push source code from VS Code to GitHub

Step 10 — Create the repository

1. Sign in to GitHub and choose New repository.
2. Name it student-k8s-demo.
3. For this classroom exercise, select Public.
4. Do not initialize the remote repository with a README; the local folder already contains the project.
5. Create the repository and copy its HTTPS URL.

Step 11 — Add .gitignore

```
.venv/
__pycache__/
.pytest_cache/
.env
```

Then run the commands below from the project directory. Replace YOUR_USERNAME with the student's GitHub username.

```
git init
```

```
git add .
git commit -m "Initial Flask application and Docker setup"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/student-k8s-demo.git
git push -u origin main
```

If Git asks for authentication, use GitHub's supported sign-in flow. Never put passwords or access tokens in source files or commit them to the repository.

Checkpoint 3: Open the repository in a browser and confirm the project files are present.

6. Add Continuous Integration with GitHub Actions

GitHub Actions can automatically check out the repository, install Python dependencies, and run the tests on each push to main and on pull requests targeting main.

Step 12 — Create .github/workflows/pipeline.yml

```
name: Student Kubernetes CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    name: Checkout and Test
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run automated tests
        run: python -m pytest -v
```

Step 13 — Commit and push the workflow

```
git add .
git commit -m "Add automated CI tests"
git push
```

Step 14 — Observe the run

1. Open the repository on GitHub.
2. Click Actions.
3. Open the Student Kubernetes CI/CD workflow run.
4. Expand each step and observe checkout, Python setup, dependency installation, and tests.

Step 15 — Demonstrate a failed test

Temporarily change the expected text in test_app.py to an incorrect value:

```
assert b"Wrong message" in response.data
```

```
git add test_app.py
git commit -m "Demonstrate a failing test"
git push
```

Show that the workflow fails. Restore the original assertion, commit, and push again. The lesson: failing tests must block later publishing and deployment steps.

7. Build and publish the Docker image automatically

Step 16 — Create a Docker Hub repository

1. Sign in to Docker Hub and create a repository named student-k8s-demo.
2. Make it Public for this introductory lab so the local cluster can pull it without registry credentials.
3. Create an access token with the minimum permissions needed to push the image.

Step 17 — Add GitHub repository secrets

In GitHub, open Settings → Secrets and variables → Actions → New repository secret. Add:

Secret name	Value
DOCKERHUB_USERNAME	Docker Hub username
DOCKERHUB_TOKEN	Docker Hub access token

Do not hard-code credentials in YAML. Keep the image repository public for this exercise; private images need an image-pull secret in Kubernetes.

Step 18 — Replace pipeline.yml with this workflow

```
name: Student Kubernetes CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    name: Checkout and Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run tests
        run: python -m pytest -v

  docker:
    name: Build and Push Docker Image
    needs: test
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/student-k8s-demo:${{ github.sha }}
```

The docker job depends on the test job. It runs only on pushes, not pull requests. The image tag uses the commit SHA, so each image is tied to a particular source revision.

```
git add .github/workflows/pipeline.yml
git commit -m "Add Docker build and publish stage"
git push
```

Confirm both jobs pass in GitHub Actions and the image appears in Docker Hub. Record the full image name and commit SHA for the Kubernetes deployment.

8. Deploy the image to Kubernetes

Key concepts: a Pod runs containers; a Deployment maintains the desired number of Pods and manages updates; a Service provides a stable network endpoint for the Pods.

Step 19 — Create k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-k8s-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: student-k8s-demo
  template:
    metadata:
      labels:
        app: student-k8s-demo
    spec:
      containers:
        - name: student-k8s-demo
          image: YOUR_DOCKERHUB_USERNAME/student-k8s-demo:REPLACE_WITH_COMMIT_SHA
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
```

Replace the image placeholder with the actual image name and SHA shown in Docker Hub or the successful GitHub Actions run. Do not use the placeholder literally.

Step 20 — Create k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: student-k8s-demo
spec:
  type: NodePort
  selector:
    app: student-k8s-demo
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
```

The Service listens on port 80, forwards to the application's port 5000, and requests NodePort 30080 for local access.

Step 21 — Apply the resources

```
kubectl config current-context
kubectl get nodes

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

kubectl rollout status deployment/student-k8s-demo
kubectl get deployments
kubectl get pods
kubectl get services
```

Use the Docker Desktop Kubernetes context. Wait until both replicas are ready. If the image is not pulled or a Pod fails, inspect the Pod details and logs before continuing.

Step 22 — Test the application

Open http://localhost:30080/ and http://localhost:30080/health. Expected home-page text: “Hello Students! Kubernetes CI/CD is working!” The health endpoint should return JSON with status healthy.

9. Demonstrate an application update

Step 23 — Change the application

Change the return statement in app.py to:

```
return "Version 2: Updated through CI/CD!"
```

Commit and push the change:

```
git add .
git commit -m "Update application message"
git push
```

After GitHub Actions passes, copy the new commit SHA and update the Deployment to use the new image tag:

```
kubectl set image deployment/student-k8s-demo student-k8s-demo=YOUR_DOCKERHUB_USERNAME/student-k8s-demo:NEW_COMMIT_SHA

kubectl rollout status deployment/student-k8s-demo
kubectl get pods
```

Replace both placeholders with the actual username and new image tag. Refresh the browser; the new message should appear.

10. Important: local deployment versus fully automated CD

In this beginner version, testing and Docker image publishing are automated, but the final Kubernetes deployment is manual. The GitHub-hosted runner runs on GitHub infrastructure and cannot normally reach Kubernetes running locally on a student's laptop.

Approach	How it works	When to use
Local cluster + manual kubectl	Actions tests and publishes the image; student deploys locally.	Best first lab; fewer credentials and networking issues.
Self-hosted runner	A runner on a trusted lab machine deploys to the local cluster after successful jobs.	Next step after the class understands the pipeline; secure runner permissions carefully.
Remote Kubernetes cluster	Actions deploys to an accessible cluster using securely managed credentials.	Cloud-based demonstration and realistic automated CD.

Teach the local workflow first. Add automatic deployment only after every student can explain each stage and debug the basic lab.

11. Common errors and troubleshooting

Problem	What to check
docker command not found	Install/start Docker Desktop; reopen the terminal.
kubectl get nodes fails	Start Kubernetes in Docker Desktop and check the current context.
Git push rejected	Check remote URL, authentication, branch, and repository setup.
GitHub Actions tests fail	Read the failed step and fix the assertion or code.
Docker login fails	Verify secret names and token permissions.
ImagePullBackOff	Check image name/tag, repository visibility, and image availability.
Pod Pending	Run kubectl describe pod and inspect events/resources.
CrashLoopBackOff	Inspect kubectl logs and application startup.
Browser cannot connect	Check Pod readiness, Service selector/endpoints, and NodePort.
Old version still appears	Verify the new tag, rollout, and browser refresh.

Useful debugging commands

```
kubectl get all
kubectl describe deployment student-k8s-demo
kubectl get pods
kubectl logs deployment/student-k8s-demo
kubectl get events --sort-by=.metadata.creationTimestamp
```

12. Teaching schedule and final checklist

Session 1 (2–3 hours)	Session 2 (2–3 hours)
Verify tools and cluster	Publish image from GitHub Actions
Create and run Flask app	Learn Pods, Deployments, and Services
Run tests locally	Deploy to Kubernetes
Build and run Docker container	Access the app in a browser
Push to GitHub	Update app and deploy a new image
Configure CI and demonstrate a failed test	Discuss self-hosted runners and remote clusters

Student demonstration checklist

- Application runs locally in VS Code.
- Automated tests pass locally.
- Docker image builds and the container runs.
- Code is pushed to GitHub.
- GitHub Actions checks out the code and runs tests.
- Docker image is published to Docker Hub.
- Kubernetes Deployment creates two ready Pods.
- Service exposes the application in the browser.
- A new code version is built and deployed successfully.

Official documentation

- GitHub Actions documentation
- Docker build documentation
- Kubernetes Deployments
- Kubernetes Services
- Kubernetes troubleshooting

Instructor note: For a first-time cohort, test the entire procedure on one machine before the class. Use public images only for this non-sensitive demo; never commit secrets, and do not use a personal production cluster for student experiments.